

p1035R0

Input range adaptors

Published Proposal, 7 May 2018

This version:

cjbdb.com.au/wg21/ranges/d1035.md

Author:

[Christopher Di Bella](#)

Audience:

LEWG

Project:

ISO JTC1/SC22/WG21: Programming Language C++

Abstract

`istream_iterator` is a great way to populate a range using some input, but it is a little bit clunky. P1035 proposes a range adaptor to eliminate the clunkiness.

Table of Contents

1	Motivation
2	<code>istream_range</code>
2.1	Interface
2.2	Co-consideration
2.3	Notes
3	Implementations
4	Exclusions
5	Acknowledgements

§ 1. Motivation

Iterators are a powerful tool in a C++ developer's toolbox. They allow us to reason about algorithms generically, and separate the implementation of an algorithm from the implementation of a data structure. An iterator range is denoted by an iterator that addresses the beginning of some sequence of data, and is terminated by some sentinel that determines when the range has been exhausted. We call a paired iterator and sentinel an 'iterator pair'.

```
// Iterator pair
copy(begin(v), end(v), std::ostream_iterator<int>{std::cout});
```

Ranges are an important abstraction over iterators, because they allow us to abstract the first element of an iterator range and the last element of an iterator range into a single entity, when the first element and last element are the beginning and end elements, respectively.

```
// Ranges abstraction
copy(v, ranges::ostream_iterator<int>{std::cout});
```

This is ideal for two reasons: firstly, it simplifies our expression from "the beginning to the end", to "the whole range". Secondly, it makes it impossible to make the following mistake:

```
// Iterator pair mistake!
copy(begin(v1), end(v2), std::ostream_iterator<int>{std::cout});
```

The use of ranges can be enjoyed today by using libraries such as `cmcstl2` or `range-v3`. Unfortunately, the Ranges TS lacks support for strict input ranges and strict output ranges. This paper proposes three input range adaptors that can be added to C++20.

§ 2. `istream_range`

`istream_iterator` is an abstraction over a `basic_istream` object, so that it may be used as though it were an input iterator. It is a great way to populate a container from the get-go, or fill a container later in its lifetime. Instead of writing a loop, such as in Listing 1.1, they can simplify their code to look like Listing 1.2.

```
// Listing 1.1
auto v = []{
    auto result = std::vector<int>{};
    for (auto i = 0; std::cin >> i;) {
        result.push_back(i);
    }
}();
// ...
for (auto i = 0; std::cin >> i;) {
    result.push_back(i);
}
```

```
// Listing 1.2
auto v = std::vector<int>(istream_iterator<int>{std::cin}, istream_iterator<int>{});
// ...
copy(istream_iterator<int>{std::cin}, istream_iterator<int>{}, back_inserter(v));
```

This is great, as `copy` is a standard algorithm that cleanly communicates that we're copying something from one range into another. There aren't any Hidden Surprises™. This is also good when writing generic code, because the generic library author does not need to care how things are inserted at the end of `v`, only that they are.

The problem with `istream_iterator` is that we need to provide a bogus `istream_iterator<T>{}` (or `default_sentinel`) every time we want to use it; this acts as the sentinel for `istream_iterator`. It is bogus, because the code is equivalent to saying "from the first element of the `istream` range until the last element of the `istream` range", but an `istream` range does not have a last element. Instead, we denote the end of an `istream` range to be when the `istream`'s `failbit` is set. This is otherwise known as the `_end-of-stream_` iterator value, but it doesn't denote a 'past-the-last element' in the same way that call to `vector<T>::end` does. Because it is the same for every range, the `_end-of-stream_` object may as well be dropped, so that we can write code that resembles Listing 1.3.

```
// Listing 1.3
auto v = std::vector<int>(istream_range<int>{std::cin});
// ...
copy(istream_range<int>{std::cin}, back_inserter(v));
```

This code is cleaner: we are implicitly saying "until our `basic_istream` object fails, insert our input into the back of `v`". There is less to read (and type!), and the code is simpler for it.

`istream_range` is a range adaptor for `istream_iterator`.

§ 2.1. Interface

All code bodies are exposition only, and are deemed to be "equivalent to".

```

template <class T, class charT = char, class traits = char_traits<charT>,
         class Distance = ptrdiff_t>
class istream_range {
    istream_iterator<T, charT, traits, Distance> iterator_{}; // exposition
public:
    using value_type = T;
    using reference = value_type&;
    using const_reference = value_type const&;
    using pointer = value_type*;
    using const_pointer = value_type const*;
    using difference_type = Distance;
    using iterator = istream_iterator<value_type, charT, traits, Distance>;
    using sentinel = iterator; // or default_sentinel
    using const_iterator = istream_iterator<value_type, charT, traits, Distance>;
    using const_sentinel = const_iterator; // or default_sentinel

    istream_range() = default;

    /// @brief Constructs an istream_range out of a previously constructed
    ///         istream_iterator.
    /// @note This constructor is useful for constructing from an istream_iterator
    ///         that has not yet reached the _end-of-stream_value (e.g. double
    ///         transform can return a useful iterator for this constructor).
    ///
    istream_range(iterator i)
        : iterator_{i}
    {}

    /// @brief Initialises the range with an input stream.
    /// @param s The input stream to be abstracted.
    /// @postcondition iterator_.in_stream == &s
    ///
    istream_range(basic_istream<charT, traits>& s)
        : iterator_{s}
    {}

    iterator begin()
    {
        return iterator_;
    }

    const_iterator begin() const
    {

```

```

        return iterator_;
    }

    auto cbegin() const
    {
        return begin();
    }

    sentinel end()
    {
        return sentinel{};
    }

    const_sentinel end() const
    {
        return sentinel{};
    }

    auto cend() const
    {
        return end();
    }

    friend bool operator==(istream_range const& a, istream_range const& b) r
    {
        return a.iterator_ == b.iterator_;
    }

    friend bool operator!=(istream_range const& a, istream_range const& b) r
    {
        return !(a == b);
    }
};

```

§ 2.2. Co-consideration

This section does not discuss `istreambuf_iterator`. Similar arguments can be made for `istreambuf_iterator`, and so P1035 advocates for them to be rangified as well.

§ 2.3. Notes

§ 3. Implementations

1. https://github.com/cjdb/cmcstl2/blob/ext-ranges/include/stl2/detail/range/istream_range.hpp
2. https://github.com/cjdb/cmcstl2/blob/ext-ranges/test/range/istream_range.cpp

§ 4. Exclusions

- Listing 1.3 teased `vector(InputRng&&)` *. P1035 does `_not_` propose a range-based constructor for containers, but a future paper will likely be presented to LEWG to do so.
- P1035 discusses ranges that can be used as input ranges only. Output ranges will be discussed in P1036, which is `_not_` a part of the Rapperswil pre-mailing.

*Terse concept syntax used for brevity only.

§ 5. Acknowledgements

I'd like to thank Morris Hafner for his review of this paper. This document was written using Markdown and rendered using [Bikeshed](#).

Finally, I'd like to thank my employer, Codeplay, for allowing me time to work on this project.